Institut National des Postes et Télécommunications (INPT) — Rabat
Agence Nationale de Réglementation des Télécommunications (ANRT)

Plateforme SMART-ICT : Connectivité

Spécialité : SMART-ICT INE3

Étude, Implémentation et Comparaison des *Graph Neural Networks* (GNN) et Synergie avec les Grands Modèles de Langage (LLM)

Cas d'étude : prédiction de la connectivité dans un réseau 5G

Réalisé par : Fatima-Ezzahra Ghanam
Houda Bouya

Encadrant(s) : *Hassan EL GHAZI*

Année universitaire 2025–2026

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrant pour ses conseils, sa disponibilité et la qualité de son suivi tout au long de ce travail. Nos remerciements vont également au corps professoral de l'INPT pour la formation reçue, ainsi qu'à l'ANRT pour le cadre scientifique et réglementaire qui inspire ce projet. Enfin, nous remercions nos familles et nos proches pour leur soutien constant.

Résumé

Ce rapport présente une étude complète des réseaux de neurones sur graphes (*Graph Neural Networks*, GNN) : fondements théoriques, architectures majeures (GCN, GAT, GraphSAGE), implémentation avec PyTorch Geometric et comparaison avec les réseaux convolutifs (CNN). Le cas d'étude porte sur un réseau 5G simplifié modélisé comme un graphe hétérogène UE–gNB : un GCN à deux couches prédit la qualité de connectivité de chaque utilisateur (mauvaise, moyenne, bonne) et est comparé à deux références sans exploitation de la topologie (règle à seuils sur la puissance reçue et perceptron multicouche). Un chapitre bonus analyse la synergie entre LLM et GNN (GraphRAG, alignement texte–graphe, enrichissement des *embeddings* par descriptions textuelles). Le code fourni est entièrement reproductible ; les valeurs numériques de la section résultats sont à renseigner après exécution.

Mots-clés : réseaux de neurones sur graphes, GCN, GAT, GraphSAGE, message passing, 5G, SINR, PyTorch Geometric, LLM, GraphRAG.

Abstract

This report presents a complete study of Graph Neural Networks (GNNs) : theoretical foundations, main architectures (GCN, GAT, GraphSAGE), an implementation with PyTorch Geometric, and a comparison with convolutional neural networks. The case study models a simplified 5G network as a UE–gNB graph, where a two-layer GCN predicts each user's connectivity quality and is benchmarked against two topology-agnostic baselines. A bonus chapter discusses the synergy between Large Language Models and GNNs (GraphRAG, text–graph alignment, LLM-enhanced node embeddings).

Keywords : graph neural networks, GCN, GAT, GraphSAGE, message passing, 5G, SINR, PyTorch Geometric, LLM, GraphRAG.

Table des matières

Remerciements i

Résumé ii

Abstract iii

Introduction Générale et Contexte du Projet 1

1 Fondements des GNN et Architectures 3
- 1.1 Introduction aux graphes dans le Deep Learning . . . 3
  - 1.1.1 Rappels sur les graphes . . . 3
  - 1.1.2 Pourquoi les CNN et RNN échouent sur les données non euclidiennes 3
- 1.2 Le cœur du GNN : l'agrégation de voisinage . . . 4
  - 1.2.1 Principe du message passing . . . 4
  - 1.2.2 Fonctions d'agrégation . . . 4
  - 1.2.3 Propriétés fondamentales . . . 5
- 1.3 Architectures majeures . . . 5
  - 1.3.1 GCN – Graph Convolutional Network . . . 5
  - 1.3.2 GAT – Graph Attention Network . . . 6
  - 1.3.3 GraphSAGE . . . 6
- 1.4 Visualisations : descriptions pour illustration . . . 7

Conclusion du chapitre . . . 8

2 Implémentation Pratique d'un GNN Simple 9
- 2.1 Modélisation du problème . . . 9
  - 2.1.1 Scénario et graphe . . . 9
  - 2.1.2 Modèle radio simplifié . . . 9
  - 2.1.3 Caractéristiques et labels . . . 10
- 2.2 Code source commenté . . . 10
  - 2.2.1 Environnement . . . 11
  - 2.2.2 Génération du scénario et construction du graphe . . . 11
  - 2.2.3 Modèle GCN à deux couches et ligne de base MLP . . . 13
  - 2.2.4 Boucle d'entraînement et évaluation . . . 14
  - 2.2.5 Tracé des courbes et de la matrice de confusion . . . 16
- 2.3 Résultats et visualisations . . . 17
  - 2.3.1 Topologie du réseau simulé . . . 17

2.3.2 Courbes d'apprentissage 18
2.3.3 Performances sur le jeu de test 19
2.4 Analyse critique et interprétation 20
2.4.1 Capacité de classification 20
2.4.2 Résultats obtenus 20
2.4.3 Surapprentissage 20
2.4.4 Impact de la topologie 20
2.4.5 Limites 21
Conclusion du chapitre 21

3 Comparatif : CNN vs GNN **22**
3.1 Tableau comparatif 22
3.2 Analyse détaillée 22
3.2.1 Le CNN comme cas particulier de GNN 23
3.2.2 Forces et limites du CNN 23
3.2.3 Forces et limites du GNN 23

4 BONUS : Synergie entre LLM et GNN **24**
4.1 Pourquoi combiner LLM et GNN ? 24
4.2 Cas d'usage majeurs 24
4.2.1 GraphRAG 24
4.2.2 Alignement texte–graphe 25
4.2.3 Amélioration des embeddings de nœuds par descriptions textuelles . 25
4.3 Architecture type de la synergie 25
4.4 Exemple de code : fusion d'embeddings textuels et structurels 26
4.5 Limites et défis 26

Conclusion et Perspectives **27**

Références Bibliographiques **28**

Table des figures

1.1 Schéma d'agrégation de voisinage (message passing) pour le nœud cible v. . 7
1.2 Architecture GCN à deux couches utilisée dans le projet (sortie : logits pour `CrossEntropyLoss`, qui intègre le softmax). 8

2.1 Graphe du réseau 5G simulé : gNB (triangles), UE colorés par classe de connectivité. 18
2.2 Courbes de loss (gauche) et d'accuracy (droite) du GCN. 18
2.3 Matrice de confusion du GCN sur le jeu de test. 19

4.1 Architecture type d'une synergie LLM + GNN pour le GraphRAG et l'alignement texte–graphe (la figure peut être réduite avec `\resizebox`). . 25

Liste des tableaux

1.1 Propriétés comparées des fonctions d'agrégation. 5
1.2 Synthèse des trois architectures. 7

2.1 Caractéristiques des nœuds (vecteur unifié de dimension 7). 10
2.2 Comparaison sur le jeu de test : moyenne ± écart-type sur 10 scénarios indépendants ($N_u = 200$, 30 UE de test par scénario, 200 époques). 19

3.1 Comparaison structurée entre réseaux convolutifs (CNN) et réseaux sur graphes (GNN). 22

Liste des codes

2.1 Installation des dépendances. 11
2.2 Génération du scénario 5G, calcul radio, labels et graphe PyG. 11
2.3 Architecture GCN (32 → 16 → 3) conforme au guide, et MLP sans topologie. 13
2.4 Entraînement (Adam, CrossEntropyLoss), sélection sur la validation, évaluation sur le test. 14
2.5 Tracé des courbes d'apprentissage et de la matrice de confusion. 16
4.1 Concaténation d'embeddings textuels (précalculés par un LLM) aux caractéristiques de nœuds avant un GCN. 26

Introduction Générale et Contexte du Projet

Contexte

L'apprentissage profond a transformé la vision par ordinateur, le traitement de la parole et du langage naturel. Ces succès reposent sur des données possédant une structure régulière : grilles de pixels pour les images, séquences pour le texte et l'audio. Or une grande partie des données du monde réel est *relationnelle* : réseaux sociaux, molécules, graphes de connaissances, réseaux routiers et, de façon centrale pour notre cursus, **réseaux de télécommunications**. Les *Graph Neural Networks* [5, 6, 7] ont été introduits pour apprendre directement sur de telles structures.

Dans un réseau mobile 5G, plusieurs équipements utilisateurs (UE) sont servis par plusieurs stations de base (gNB). La qualité de service d'un UE dépend de sa propre mesure radio (puissance reçue, SINR, distance), mais aussi de la charge de sa cellule et de l'interférence créée par les cellules voisines. Le réseau possède donc une structure relationnelle naturelle, que les GNN peuvent exploiter [15]. La pertinence de ce sujet est d'autant plus forte que la régulation et le déploiement de la 5G (domaine de l'ANRT) exigent des outils d'analyse de la qualité de couverture et de connectivité.

Problématique

La question centrale du projet est la suivante :

> *Un modèle qui tient compte de la topologie du réseau peut-il mieux prédire la qualité de connectivité d'un utilisateur qu'un modèle ou une règle classique n'utilisant que ses mesures individuelles ?*

Objectifs

- Comprendre pourquoi les CNN et RNN sont inadaptés aux données non euclidiennes et en quoi les GNN y remédient ;
- Maîtriser le mécanisme de *message passing* et les architectures GCN, GAT et GraphSAGE ;
- Implémenter un GCN à deux couches avec PyTorch Geometric sur un scénario 5G synthétique, avec une évaluation sans fuite de données ;

— Comparer rigoureusement CNN et GNN ;

— Explorer, en bonus, la synergie LLM + GNN.

Plan du rapport

Le **Chapitre 1** pose les fondements des GNN et leurs architectures majeures. Le **Chapitre 2** détaille l'implémentation d'un GNN simple pour la connectivité 5G, avec code, courbes d'apprentissage et analyse critique. Le **Chapitre 3** propose un comparatif structuré CNN vs GNN. Le **Chapitre 4** (bonus) étudie la synergie entre LLM et GNN. Une conclusion et des perspectives clôturent le document, suivies des références au format IEEE.

Chapitre 1

Fondements des GNN et Architectures

1.1 Introduction aux graphes dans le Deep Learning

1.1.1 Rappels sur les graphes

Un graphe est noté $G = (V, E)$, où V est l'ensemble des N nœuds et $E \subseteq V \times V$ l'ensemble des arêtes. La structure est encodée par la matrice d'adjacence $\boldsymbol{A} \in \{0,1\}^{N\times N}$ ($A_{vu} = 1$ si l'arête (u, v) existe). Chaque nœud v porte un vecteur de caractéristiques $\boldsymbol{x}_v \in \mathbb{R}^F$; on note $\boldsymbol{X} \in \mathbb{R}^{N\times F}$ la matrice des caractéristiques. Le voisinage de v est $\mathcal{N}(v) = \{u : (u, v) \in E\}$ et son degré $d_v = |\mathcal{N}(v)|$. Des attributs d'arête $\boldsymbol{e}_{uv}$ peuvent s'ajouter.

1.1.2 Pourquoi les CNN et RNN échouent sur les données non euclidiennes

Les CNN [10] exploitent trois hypothèses : (i) les données vivent sur une **grille régulière** ; (ii) chaque position possède un **voisinage de taille fixe** et ordonné (noyau 3×3 par exemple) ; (iii) il existe une notion de **translation** permettant de partager les poids. Les graphes violent ces hypothèses [6] :

— **Absence d'ordre canonique** : il n'y a ni "gauche/droite" ni "haut/bas". Numéroter les nœuds différemment (permutation $\boldsymbol{P}$) ne change pas le graphe, alors que cela change l'entrée d'un réseau classique ;

— **Degré variable** : un nœud peut avoir 1 ou 10 000 voisins, un noyau de taille fixe n'a donc pas de sens ;

— **Taille variable** : les graphes d'un jeu de données n'ont pas le même nombre de nœuds, contrairement à des images redimensionnées ;

— **Les RNN** supposent un ordre séquentiel total ; linéariser un graphe en séquence impose un ordre arbitraire et détruit les relations locales.

Un MLP appliqué à la matrice d'adjacence aplatie serait quant à lui dépendant de l'ordre des nœuds et de la taille N, et ne partagerait pas les paramètres entre nœuds. Les GNN résolvent ces difficultés en construisant des opérateurs **locaux**, à **poids partagés** et **équivariants aux permutations**.

1.2 Le cœur du GNN : l'agrégation de voisinage

1.2.1 Principe du message passing

Le cadre général du *Message Passing Neural Network* (MPNN) [4] décrit une couche de GNN en trois étapes : **message**, **agrégation**, **mise à jour**. En notant $\boldsymbol{h}_v^{(l)}$ la représentation du nœud v à la couche l (avec $\boldsymbol{h}_v^{(0)} = \boldsymbol{x}_v$) :

$$\boldsymbol{m}_v^{(l)} = \underset{u \in \mathcal{N}(v)}{\mathrm{AGG}} \Big(\overset{(l)}{\mathrm{MSG}} \left(\boldsymbol{h}_u^{(l)}, \boldsymbol{h}_v^{(l)}, \boldsymbol{e}_{uv} \right) \Big), \qquad \boldsymbol{h}_v^{(l+1)} = \overset{(l)}{\mathrm{UPDATE}} \left(\boldsymbol{h}_v^{(l)}, \boldsymbol{m}_v^{(l)} \right). \tag{1.1}$$

Dans la forme simplifiée du guide du projet, cela s'écrit

$$\boldsymbol{h}_v^{(l+1)} = \sigma\Big(\boldsymbol{W}^{(l)} \cdot \mathrm{AGG} \left(\{ \boldsymbol{h}_u^{(l)} : u \in \mathcal{N}(v) \} \right) \Big), \tag{1.2}$$

où $\boldsymbol{W}^{(l)}$ est une matrice de poids partagée par tous les nœuds et σ une non-linéarité (ReLU). Après L couches, la représentation d'un nœud intègre l'information de son voisinage à L sauts : c'est son *champ récepteur*.

1.2.2 Fonctions d'agrégation

Pour être valide, AGG doit être **invariante aux permutations** de ses arguments (les voisins n'ont pas d'ordre). Les choix usuels sont :

$$\text{Somme :} \quad \boldsymbol{m}_v = \sum_{u \in \mathcal{N}(v)} \boldsymbol{h}_u, \tag{1.3}$$

$$\text{Moyenne :} \quad \boldsymbol{m}_v = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \boldsymbol{h}_u, \tag{1.4}$$

$$\text{Maximum :} \quad \boldsymbol{m}_v = \max_{u \in \mathcal{N}(v)} \boldsymbol{h}_u \quad \text{(composante par composante).} \tag{1.5}$$

Table 1.1 – Propriétés comparées des fonctions d'agrégation.

Agrégation	Avantages	Limites
Somme	Préserve la multiplicité du voisinage (information sur le degré) ; expressivité maximale parmi les trois [8].	Amplitude dépendant du degré : risque d'instabilité numérique sur des nœuds très connectés.
Moyenne	Normalise par le degré : stable, adaptée à des degrés hétérogènes.	Perd l'information de cardinalité (deux multi-ensembles proportionnels sont indiscernables).
Maximum	Capte les signaux saillants ; robuste aux valeurs moyennes bruitées.	Perd la multiplicité et la distribution ; peu sensible aux petites variations.

1.2.3 Propriétés fondamentales

— **Équivariance aux permutations** : pour toute matrice de permutation $\boldsymbol{P}$, $f(\boldsymbol{P}\boldsymbol{X}\boldsymbol{P}^\top, \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^\top) = \boldsymbol{P}f(\boldsymbol{X}, \boldsymbol{A})$ pour une couche de nœud. Pour une tâche de graphe entier, on ajoute une lecture globale (*readout*) invariante.

— **Localité et partage de paramètres** : le nombre de paramètres est indépendant de N.

— **Expressivité** : les GNN de type MPNN sont au plus aussi discriminants que le test de Weisfeiler–Leman [8].

— **Sur-lissage (*over-smoothing*)** : un empilement excessif de couches rend les représentations des nœuds quasi identiques [11]. Quelques couches (2 à 3) suffisent généralement.

— **Goulot d'étranglement (*over-squashing*)** : l'information de voisins lointains est compressée dans un vecteur de taille fixe [12].

1.3 Architectures majeures

1.3.1 GCN – Graph Convolutional Network

Kipf et Welling [1] simplifient la convolution spectrale sur graphes [9] en une règle de propagation du premier ordre. Avec $\tilde{\boldsymbol{A}} = \boldsymbol{A} + \boldsymbol{I}_N$ (boucles propres) et $\tilde{\boldsymbol{D}}_{ii} = \sum_j \tilde{A}_{ij}$:

$$\boldsymbol{H}^{(l+1)} = \sigma\Big(\tilde{\boldsymbol{D}}^{-1/2}\,\tilde{\boldsymbol{A}}\,\tilde{\boldsymbol{D}}^{-1/2}\,\boldsymbol{H}^{(l)}\boldsymbol{W}^{(l)}\Big). \tag{1.6}$$

Au niveau d'un nœud, cela correspond à une moyenne pondérée symétriquement normalisée :

$$\boldsymbol{h}_v^{(l+1)} = \sigma\Big(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{d}_v\,\tilde{d}_u}}\, \boldsymbol{W}^{(l)} \boldsymbol{h}_u^{(l)} \Big). \tag{1.7}$$

Points forts : simple, peu de paramètres, efficace. **Limites** : poids d'agrégation fixés par la topologie (tous les voisins sont traités de façon similaire), nature transductive de l'article original, sensibilité au sur-lissage.

1.3.2 GAT – Graph Attention Network

GAT [2] apprend des coefficients d'attention α_{vu} pour pondérer les voisins :

$$e_{vu} = \mathrm{LeakyReLU}\Big(\boldsymbol{a}^\top [\boldsymbol{W}\boldsymbol{h}_v \,\|\, \boldsymbol{W}\boldsymbol{h}_u] \Big), \tag{1.8}$$

$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in \mathcal{N}(v)\cup\{v\}} \exp(e_{vk})}, \tag{1.9}$$

$$\boldsymbol{h}_v^{(l+1)} = \sigma\Big(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \alpha_{vu}\, \boldsymbol{W}\boldsymbol{h}_u^{(l)} \Big), \tag{1.10}$$

avec plusieurs têtes d'attention concaténées (ou moyennées en sortie). Le modèle s'adapte à l'importance de chaque voisin et se prête à l'interprétation ; son coût est plus élevé. Une variante, GATv2, corrige une limitation d'expressivité de l'attention statique [13].

1.3.3 GraphSAGE

GraphSAGE [3] est conçu pour l'apprentissage **inductif** : il apprend des fonctions d'agrégation applicables à des nœuds ou graphes jamais vus, avec échantillonnage d'un nombre fixe de voisins par nœud (passage à l'échelle) :

$$\boldsymbol{h}_v^{(l+1)} = \sigma\Big(\boldsymbol{W}^{(l)} \cdot \Big[\boldsymbol{h}_v^{(l)} \,\|\, \underset{u \in \mathcal{S}(v)}{\mathrm{AGG}}\, \boldsymbol{h}_u^{(l)} \Big] \Big), \qquad \boldsymbol{h}_v^{(l+1)} \leftarrow \frac{\boldsymbol{h}_v^{(l+1)}}{\|\boldsymbol{h}_v^{(l+1)}\|_2}, \tag{1.11}$$

où $\mathcal{S}(v) \subseteq \mathcal{N}(v)$ est l'échantillon de voisins et $\mathrm{AGG} \in \{\text{moyenne}, \text{LSTM}, \text{pooling max}\}$. La concaténation sépare la contribution du nœud lui-même de celle du voisinage.

Table 1.2 – Synthèse des trois architectures.

	GCN [1]	**GAT [2]**	**GraphSAGE [3]**
Pondération des voisins	Fixe (degrés)	Apprise (attention)	Uniforme selon l'agrégateur
Régime	Transductif (en pratique aussi inductif)	Transductif / inductif	Inductif
Coût	Faible	Modéré à élevé	Contrôlé par l'échantillonnage
Classe PyG	`GCNConv`	`GATConv`	`SAGEConv`

1.4 Visualisations : descriptions pour illustration

Description du Schéma 1 – Agrégation de voisinage (Message Passing)

Placement : Figure 1.1, Section 1.4.

Contenu : Au centre, un nœud cible v (cercle bleu foncé) ; autour, quatre voisins $u_1, \ldots, u_4$ (cercles bleu clair), chacun portant son vecteur $\boldsymbol{h}_{u_i}^{(l)}$. Des flèches orientées $u_i \to v$ représentent les *messages* $\boldsymbol{m}_{u_i \to v} = \mathrm{MSG}(\boldsymbol{h}_{u_i}, \boldsymbol{h}_v)$. Ces flèches convergent vers une boîte rouge "AGG (somme, moyenne ou max)", qui produit $\boldsymbol{m}_v$. Une seconde boîte "UPDATE ($\boldsymbol{W}$, σ)" combine $\boldsymbol{m}_v$ et $\boldsymbol{h}_v^{(l)}$ pour donner $\boldsymbol{h}_v^{(l+1)}$. Une légende indique que le voisin u_4 est lui-même entouré de ses propres voisins en gris : à la couche suivante, ils influenceront v via u_4 (champ récepteur à 2 sauts).

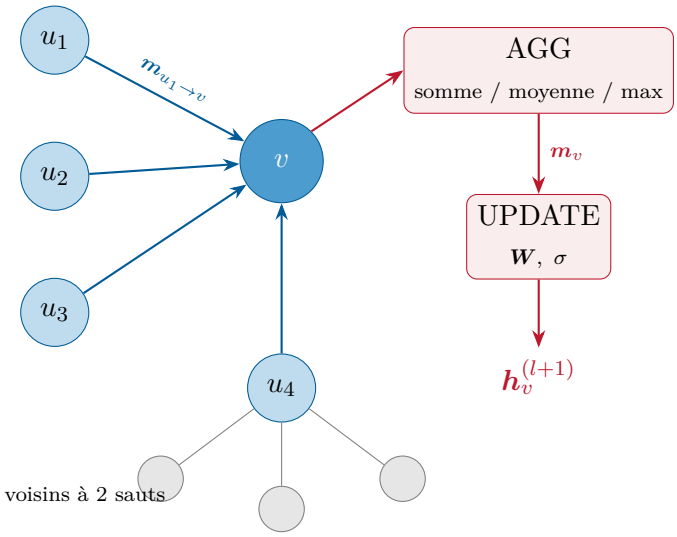


Figure 1.1 – Schéma d'agrégation de voisinage (message passing) pour le nœud cible v.

Description du Schéma 2 – Architecture GCN globale

Placement : Figure 1.2, Section 1.4.

Contenu : Un flux de gauche à droite : (1) bloc "Graphe d'entrée" avec $\boldsymbol{X} \in \mathbb{R}^{N \times F}$ et $\boldsymbol{A}$; (2) bloc "GCN_1 $(F \to 32)$" suivi de "ReLU" (et optionnellement "Dropout") ; (3) bloc "GCN_2 $(32 \to 16)$" suivi de "ReLU" ; (4) bloc "Linéaire $(16 \to C)$" ; (5) bloc "Softmax" donnant C probabilités par nœud. Sous les blocs GCN, une bande indique que la matrice normalisée $\hat{\boldsymbol{A}} = \tilde{\boldsymbol{D}}^{-1/2}\tilde{\boldsymbol{A}}\tilde{\boldsymbol{D}}^{-1/2}$ est réutilisée à chaque couche.

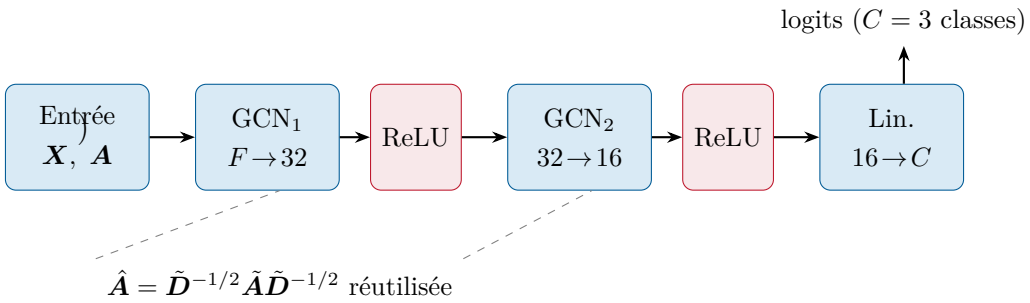


Figure 1.2 – Architecture GCN à deux couches utilisée dans le projet (sortie : logits pour `CrossEntropyLoss`, qui intègre le softmax).

Conclusion du chapitre

Les GNN généralisent la convolution aux domaines irréguliers par l'agrégation locale et invariante des voisins. GCN, GAT et GraphSAGE diffèrent par la pondération des voisins, le régime (transductif ou inductif) et le coût. Le chapitre suivant applique un GCN à un problème concret de réseau 5G.

Chapitre 2

Implémentation Pratique d'un GNN Simple

Ce chapitre implémente le projet décrit dans le guide étudiant *"GNN pour la prédiction et l'optimisation de la connectivité dans un réseau 5G"*. La question est de savoir si l'exploitation de la topologie permet de prédire la classe de connectivité (mauvaise, moyenne, bonne) mieux qu'une règle classique.

2.1 Modélisation du problème

2.1.1 Scénario et graphe

Une zone 2D de 1000×1000 m contient N_g gNB et N_u UE placés aléatoirement. Nous construisons un **graphe hétérogène mais encodé dans un seul graphe homogène** (un indicateur de type est ajouté aux caractéristiques) :

- **Nœuds** : les UE et les gNB ;
- **Arêtes UE–gNB** (bipartites, rendues non orientées) : un UE est relié à un gNB si la puissance reçue dépasse un seuil de couverture $P_{\min} = -100$ dBm ; le gNB servant l'UE (meilleure puissance) est toujours relié ;
- **Arêtes gNB–gNB** : graphe complet entre stations (liaisons voisines / lien d'interférence), ce qui permet à l'information de charge des cellules voisines de se propager ;
- **Cible** : la classe de connectivité des nœuds UE.

Avec deux couches de convolution, un UE reçoit l'information de sa cellule (1 saut) puis, via les arêtes gNB–gNB, celle des cellules voisines (2 sauts), tandis que la cellule agrège elle-même les UE qu'elle sert.

2.1.2 Modèle radio simplifié

Le but est pédagogique : produire des mesures cohérentes, non reproduire un modèle normatif (voir [16] pour les modèles réalistes). La distance est $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ et

$$P_r^{(ij)} = P_t - PL(d_{ij}) + X_\sigma, \qquad PL(d) = PL_0 + 10\, n \log_{10}\big(d/d_0\big), \tag{2.1}$$

avec $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ (ombrage log-normal). L'UE i est servi par $s(i) = \arg\max_j P_r^{(ij)}$ et son SINR (linéaire) vaut

$$\mathrm{SINR}_i = \frac{p_{i,s(i)}}{\sum_{k \neq s(i)} \rho_k \, p_{ik} + N_0}, \qquad p_{ij} = 10^{P_r^{(ij)}/10} \ (\mathrm{mW}), \tag{2.2}$$

où $\rho_k \in [0{,}05; 1]$ est la charge du gNB k (trafic servi divisé par sa capacité) : un gNB plus chargé interfère davantage. **Paramètres utilisés** : $P_t = 43\,\mathrm{dBm}$, $PL_0 = 43{,}3\,\mathrm{dB}$ ($d_0 = 1\,\mathrm{m}$, ordre de grandeur de l'espace libre à 3,5 GHz), $n = 3{,}5$, $\sigma = 4\,\mathrm{dB}$, bruit $-94\,\mathrm{dBm}$, capacité 400 Mbps. Ces valeurs sont des hypothèses de travail modifiables.

2.1.3 Caractéristiques et labels

Table 2.1 – Caractéristiques des nœuds (vecteur unifié de dimension 7).

Indice	Variable	Description / normalisation
0	type	0 = UE, 1 = gNB
1–2	x, y	Position, divisée par 1000 m
3	trafic / degré	UE : trafic demandé /20 Mbps ; gNB : fraction d'UE couverts
4	P_r	UE : puissance reçue du gNB servant, $(P_r + 110)/60$
5	distance	UE : distance au gNB servant, /500 m
6	charge	gNB : charge ρ_k ; 0 pour les UE

Prévention de la fuite de données

Le **SINR n'est pas fourni en entrée** : il sert à définir les labels ($y = 0$ si SINR $< 5\,\mathrm{dB}$, $y = 1$ si $5 \leq \mathrm{SINR} < 15\,\mathrm{dB}$, $y = 2$ sinon ; classes 1, 2, 3 du guide, indexées depuis 0 pour PyTorch). Fournir le SINR rendrait la tâche triviale. Le SINR dépend de l'interférence des cellules voisines (donc de leur charge), information accessible à l'UE uniquement par la topologie : c'est ce qui motive le GNN. Les constantes de normalisation sont fixes (physiques), donc aucune statistique du jeu de test n'est utilisée. Les seuils de 5 et 15 dB sont conservés du guide ; ils correspondent grosso modo à une liaison difficile, exploitable et confortable, et peuvent être ajustés s'ils sont justifiés.

2.2 Code source commenté

2.2.1 Environnement

```
pip install torch torch-geometric numpy pandas matplotlib scikit-learn
```

Code 2.1 – Installation des dépendances.

2.2.2 Génération du scénario et construction du graphe

```
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from torch_geometric.data import Data
from torch_geometric.nn import GCNConv
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt

SEED = 42
np.random.seed(SEED); torch.manual_seed(SEED)

# ---- Parametres du scenario (hypotheses pedagogiques) ----
AREA, N_GNB, N_UE = 1000.0, 5, 200
PT, PL0, D0, N_EXP, SIGMA = 43.0, 43.3, 1.0, 3.5, 4.0   # dBm, dB, m, -, dB
NOISE_DBM, PR_MIN, CAP = -94.0, -100.0, 400.0           # dBm, dBm, Mbps
SINR_THRESH = [5.0, 15.0]   # classes: <5 | 5-15 | >=15 dB (0,1,2)

def generate_scenario(seed=SEED):
    rng = np.random.default_rng(seed)
    gnb_xy = rng.uniform(100, AREA - 100, (N_GNB, 2))
    ue_xy = rng.uniform(0, AREA, (N_UE, 2))
    traffic = rng.gamma(2.0, 2.5, N_UE).clip(0.5, 20.0)          # Mbps (moyenne ~5)

    # Distances UE-gNB et puissance recue (modele log-distance + ombrage)
    d = np.linalg.norm(ue_xy[:, None, :] - gnb_xy[None, :, :], axis=2).clip(min=10.0)
    pl = PL0 + 10 * N_EXP * np.log10(d / D0)
    pr = PT - pl + rng.normal(0, SIGMA, d.shape)                 # dBm, shape (N_UE, N_GNB)

    # Association au meilleur gNB et charge des cellules
    idx = np.arange(N_UE)
```

```
    serving = pr.argmax(axis=1)
    load = np.array([traffic[serving == k].sum() / CAP for k in range
        (N_GNB)])
    load = load.clip(0.05, 1.0)

    # SINR : interference = somme des autres cellules ponderee par
        leur charge
    p_lin = 10 ** (pr / 10)
    signal = p_lin[idx, serving]
    interf = (p_lin * load[None, :]).sum(axis=1) - signal * load[
        serving]
    noise = 10 ** (NOISE_DBM / 10)
    sinr = 10 * np.log10(signal / (interf + noise))
    labels = np.digitize(sinr, SINR_THRESH)                          # 0,
         1, 2
    return dict(gnb_xy=gnb_xy, ue_xy=ue_xy, traffic=traffic, d=d, pr=
        pr,
                serving=serving, load=load, sinr=sinr, labels=labels)

def build_graph(sc):
    n_ue, n_gnb = N_UE, N_GNB
    idx = np.arange(n_ue)
    cover = sc["pr"] > PR_MIN
    cover[idx, sc["serving"]] = True                                 # le
         gNB servant est toujours relie
    ue_i, g_i = np.nonzero(cover)
    a, b = ue_i, n_ue + g_i                                          #
        gNB indexes apres les UE
    ug = np.stack([np.concatenate([a, b]), np.concatenate([b, a])])
          # UE<->gNB
    gg = np.array([[n_ue + i, n_ue + j] for i in range(n_gnb)
                   for j in range(n_gnb) if i != j]).T
                                        # gNB<->gNB
    edge_index = torch.tensor(np.concatenate([ug, gg], axis=1), dtype
        =torch.long)

    # Matrice de caracteristiques (dim 7) : [type, x, y, trafic/degre
        , Pr, dist, charge]
    X = np.zeros((n_ue + n_gnb, 7), dtype=np.float32)
    X[:n_ue, 1:3] = sc["ue_xy"] / AREA
    X[:n_ue, 3] = sc["traffic"] / 20.0
    X[:n_ue, 4] = (sc["pr"][idx, sc["serving"]] + 110.0) / 60.0
    X[:n_ue, 5] = sc["d"][idx, sc["serving"]] / 500.0
    X[n_ue:, 0] = 1.0
    X[n_ue:, 1:3] = sc["gnb_xy"] / AREA
    X[n_ue:, 3] = cover.sum(axis=0) / n_ue
    X[n_ue:, 6] = sc["load"]
```

```
68
69      y = np.zeros(n_ue + n_gnb, dtype=np.int64); y[:n_ue] = sc["labels
            "]
70
71      # Split 70/15/15 sur les noeuds UE uniquement (graine fixe)
72      perm = np.random.default_rng(SEED).permutation(n_ue)
73      n_tr, n_va = int(0.70 * n_ue), int(0.15 * n_ue)
74      def mask(ids):
75          m = torch.zeros(n_ue + n_gnb, dtype=torch.bool); m[ids] =
                True; return m
76      return Data(x=torch.tensor(X), edge_index=edge_index, y=torch.
            tensor(y),
77                  train_mask=mask(perm[:n_tr]), val_mask=mask(perm[n_tr
                        :n_tr + n_va]),
78                  test_mask=mask(perm[n_tr + n_va:]))
79
80 sc = generate_scenario()
81 data = build_graph(sc)
82 print(data)
83 print("Repartition des classes (UE):", np.bincount(sc["labels"],
       minlength=3))
84
85 # Export tabulaire au format de l'annexe A du guide (UE, gNB,
       distance, Pr, SINR, trafic, charge, classe)
86 rows = [(i + 1, j + 1, sc["d"][i, j], sc["pr"][i, j], sc["sinr"][i]
       if j == sc["serving"][i] else np.nan,
87           sc["traffic"][i], sc["load"][j], sc["labels"][i] + 1)
88          for i in range(N_UE) for j in range(N_GNB) if sc["pr"][i, j]
                > PR_MIN]
89 pd.DataFrame(rows, columns=["UE", "gNB", "distance_m", "Pr_dBm", "
       SINR_dB",
90                              "trafic_Mbps", "charge", "classe"]).
                                    to_csv("dataset_5g.csv", index=False)
```

Code 2.2 – Génération du scénario 5G, calcul radio, labels et graphe PyG.

2.2.3 Modèle GCN à deux couches et ligne de base MLP

```
1 class GCN(torch.nn.Module):
2     """Entree -> GCN(32) -> ReLU -> GCN(16) -> ReLU -> Lineaire -> 3
          classes."""
3     def __init__(self, in_ch, hidden=32, out_ch=3, p_drop=0.3):
4         super().__init__()
5         self.conv1 = GCNConv(in_ch, hidden)
6         self.conv2 = GCNConv(hidden, 16)
7         self.fc = torch.nn.Linear(16, out_ch)
```

```
        self.p = p_drop

    def forward(self, x, edge_index):
        x = F.relu(self.conv1(x, edge_index))
        x = F.dropout(x, p=self.p, training=self.training)
        x = F.relu(self.conv2(x, edge_index))
        return self.fc(x)                       # logits (softmax
            inclus dans la loss)

class MLP(torch.nn.Module):
    """Meme profondeur/largeur, mais SANS information de voisinage (
        ablation)."""
    def __init__(self, in_ch, hidden=32, out_ch=3, p_drop=0.3):
        super().__init__()
        self.l1, self.l2 = torch.nn.Linear(in_ch, hidden), torch.nn.
            Linear(hidden, 16)
        self.fc, self.p = torch.nn.Linear(16, out_ch), p_drop

    def forward(self, x, edge_index=None):
        x = F.relu(self.l1(x))
        x = F.dropout(x, p=self.p, training=self.training)
        return self.fc(F.relu(self.l2(x)))
```

Code 2.3 – Architecture GCN ($32 \to 16 \to 3$) conforme au guide, et MLP sans topologie.

2.2.4 Boucle d'entraînement et évaluation

```
def accuracy(logits, y, mask):
    return (logits[mask].argmax(1) == y[mask]).float().mean().item()

def fit(model, data, epochs=200, lr=0.01, wd=5e-4):
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=wd
        )
    hist = {k: [] for k in ["tr_loss", "va_loss", "tr_acc", "va_acc"
        ]}
    best_va, best_state = -1.0, None
    for ep in range(epochs):
        model.train(); opt.zero_grad()
        out = model(data.x, data.edge_index)
        loss = F.cross_entropy(out[data.train_mask], data.y[data.
            train_mask])
        loss.backward(); opt.step()

        model.eval()
        with torch.no_grad():
            out = model(data.x, data.edge_index)
```

```
            va_loss = F.cross_entropy(out[data.val_mask], data.y[data.val_mask]).item()
            tr_acc, va_acc = accuracy(out, data.y, data.train_mask), accuracy(out, data.y, data.val_mask)
        for k, v in zip(hist, [loss.item(), va_loss, tr_acc, va_acc]):
            hist[k].append(v)
        if va_acc > best_va:                    # le test n'est JAMAIS utilise pour choisir
            best_va = va_acc
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
    model.load_state_dict(best_state)
    return hist

@torch.no_grad()
def evaluate(model, data):
    model.eval()
    pred = model(data.x, data.edge_index).argmax(1)
    m = data.test_mask
    return (pred[m] == data.y[m]).float().mean().item(), data.y[m].numpy(), pred[m].numpy()

# ---- GCN ----
gcn = GCN(data.num_node_features)
hist_gcn = fit(gcn, data)
acc_gcn, y_true, y_pred = evaluate(gcn, data)

# ---- Baseline 1 : MLP sans topologie ----
mlp = MLP(data.num_node_features)
hist_mlp = fit(mlp, data)
acc_mlp, _, _ = evaluate(mlp, data)

# ---- Baseline 2 : regle a seuils sur la puissance recue (parametres ajustes sur le TRAIN) ----
pr_dbm = data.x[:, 4].numpy() * 60.0 - 110.0
tr, te = data.train_mask.numpy(), data.test_mask.numpy()
y_np = data.y.numpy()
best_rule, best_acc = None, -1.0
for t1 in np.arange(-100, -60, 1.0):
    for t2 in np.arange(t1 + 1, -55, 1.0):
        a = (np.digitize(pr_dbm[tr], [t1, t2]) == y_np[tr]).mean()
        if a > best_acc: best_rule, best_acc = (t1, t2), a
acc_rule = (np.digitize(pr_dbm[te], list(best_rule)) == y_np[te]).mean()

```

```
print(f"Test accuracy -> GCN: {acc_gcn:.3f} | MLP: {acc_mlp:.3f} |
    Regle Pr: {acc_rule:.3f}")
print(classification_report(y_true, y_pred, target_names=["mauvaise",
     "moyenne", "bonne"], zero_division=0))
```

Code 2.4 – Entraînement (Adam, CrossEntropyLoss), sélection sur la validation, évaluation sur le test.

2.2.5 Tracé des courbes et de la matrice de confusion

```
import os; os.makedirs("figures", exist_ok=True)

def plot_curves(hist, name):
    ep = range(1, len(hist["tr_loss"]) + 1)
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
    ax[0].plot(ep, hist["tr_loss"], label="Entrainement"); ax[0].plot
       (ep, hist["va_loss"], label="Validation")
    ax[0].set_xlabel("Epoque"); ax[0].set_ylabel("Loss (cross-entropy
       )"); ax[0].legend(); ax[0].grid(alpha=.3)
    ax[1].plot(ep, hist["tr_acc"], label="Entrainement"); ax[1].plot(
       ep, hist["va_acc"], label="Validation")
    ax[1].set_xlabel("Epoque"); ax[1].set_ylabel("Accuracy"); ax[1].
       legend(); ax[1].grid(alpha=.3)
    plt.tight_layout(); plt.savefig(f"figures/courbes-{name}.png",
       dpi=200); plt.close()

plot_curves(hist_gcn, "gcn")
plot_curves(hist_mlp, "mlp")

cm = confusion_matrix(y_true, y_pred, labels=[0, 1, 2])
fig, ax = plt.subplots(figsize=(4.2, 3.8))
im = ax.imshow(cm, cmap="Blues"); plt.colorbar(im)
ax.set_xticks(range(3)); ax.set_yticks(range(3))
ax.set_xticklabels(["mauv.", "moy.", "bonne"]); ax.set_yticklabels(["
    mauv.", "moy.", "bonne"])
ax.set_xlabel("Classe predite"); ax.set_ylabel("Classe reelle")
for i in range(3):
    for j in range(3):
        ax.text(j, i, cm[i, j], ha="center", va="center", color="
           black")
plt.tight_layout(); plt.savefig("figures/confusion-gcn.png", dpi=200)
    ; plt.close()

# Visualisation du graphe (positions reelles, couleur = classe)
fig, ax = plt.subplots(figsize=(5.5, 5.5))
ei = data.edge_index.numpy(); pos = data.x[:, 1:3].numpy() * AREA
```

```
for s, t in ei.T:
    if s < N_UE and t >= N_UE: ax.plot(*zip(pos[s], pos[t]), c="
        lightgray", lw=.4, zorder=1)
ax.scatter(pos[:N_UE, 0], pos[:N_UE, 1], c=sc["labels"], cmap="RdYlGn
    ", s=14, zorder=2)
ax.scatter(pos[N_UE:, 0], pos[N_UE:, 1], marker="^", c="black", s
    =120, zorder=3, label="gNB")
ax.set_xlabel("x (m)"); ax.set_ylabel("y (m)"); ax.legend()
plt.tight_layout(); plt.savefig("figures/graphe-5g.png", dpi=200);
    plt.close()
```

Code 2.5 – Tracé des courbes d'apprentissage et de la matrice de confusion.

Utilisation. Exécuter les blocs 2.2 à 2.5 dans un seul notebook ou script ; les figures sont écrites dans `figures/` et sont automatiquement incluses par ce document à la recompilation.

2.3 Résultats et visualisations

2.3.1 Topologie du réseau simulé

Description – Figure ??

Placement : fichier `graphe-5g.png`. Nuage de points dans la zone 1000×1000 m : triangles noirs pour les 5 gNB, petits cercles colorés du rouge (classe 0, mauvaise) au vert (classe 2, bonne) pour les UE, et traits gris clairs pour les arêtes UE–gNB. On attend des UE verts proches des gNB et des UE rouges aux frontières entre cellules, où l'interférence est maximale.

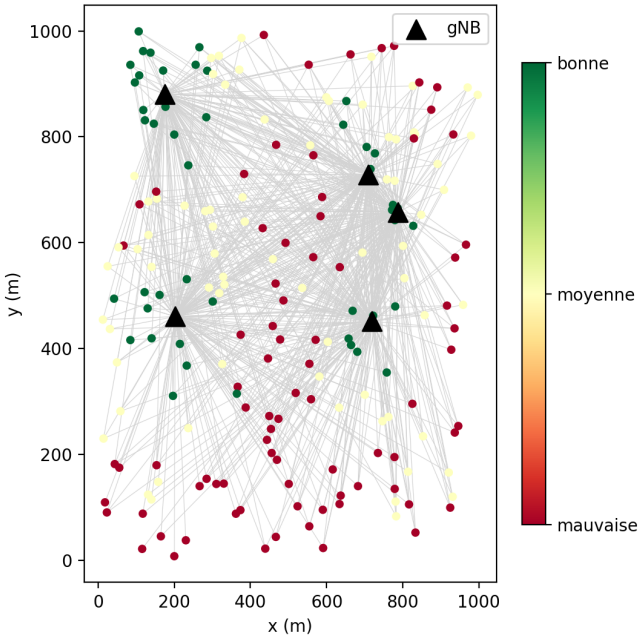


Figure 2.1 – Graphe du réseau 5G simulé : gNB (triangles), UE colorés par classe de connectivité.

2.3.2 Courbes d'apprentissage

Description – Figure 2.2

Placement : fichier `courbes-gcn.png`. Deux sous-graphes. *Gauche* : évolution de la loss d'entraînement et de validation en fonction de l'époque (axe x : 1 à 200). *Droite* : accuracy d'entraînement et de validation. **Comportement attendu** : la loss d'entraînement décroît de $\approx \ln 3 \approx 1{,}10$ (hasard) vers une valeur faible ; la loss de validation suit d'abord, puis se stabilise ou remonte légèrement en cas de surapprentissage ; l'accuracy de validation plafonne alors que celle d'entraînement continue de croître.

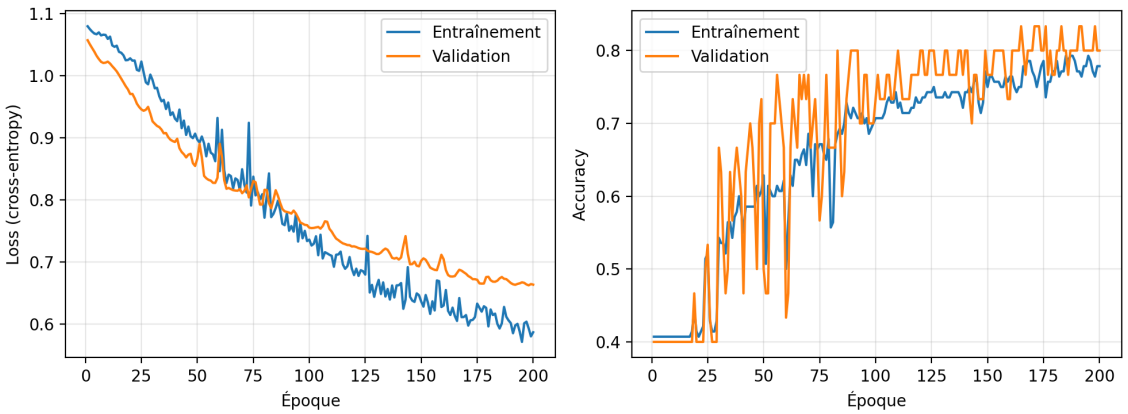


Figure 2.2 – Courbes de loss (gauche) et d'accuracy (droite) du GCN.

Instructions de tracé

Les courbes sont produites par la fonction `plot_curves` (Code 2.5). Pour comparer plusieurs modèles, tracer `hist_gcn["va_acc"]` et `hist_mlp["va_acc"]` sur le même axe. Pour une analyse statistiquement solide, répéter l'expérience avec plusieurs graines (par exemple 10), puis tracer la moyenne ± l'écart-type ; `plt.fill_between` permet d'afficher la bande d'incertitude.

2.3.3 Performances sur le jeu de test

Table 2.2 – Comparaison sur le jeu de test : moyenne ± écart-type sur 10 scénarios indépendants ($N_u = 200$, 30 UE de test par scénario, 200 époques).

Méthode	Accuracy test	F1 macro	Utilise la topologie
Règle à seuils sur P_r	$72{,}0 \pm 5{,}6\,\%$	$67{,}1 \pm 6{,}2\,\%$	Non
MLP (même largeur)	$73{,}3 \pm 10{,}0\,\%$	$66{,}7 \pm 12{,}6\,\%$	Non
GCN (2 couches)	$65{,}0 \pm 12{,}8\,\%$	$53{,}5 \pm 18{,}3\,\%$	Oui
GCN sans arêtes gNB–gNB	$62{,}0 \pm 13{,}2\,\%$	$49{,}2 \pm 18{,}0\,\%$	Partielle

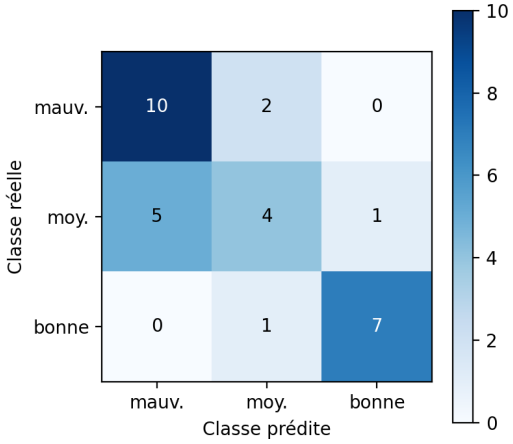


Figure 2.3 – Matrice de confusion du GCN sur le jeu de test.

Rigueur expérimentale

Les figures et le tableau proviennent du script `experience_gnn_5g_numpy.py`, qui réimplémente en NumPy exactement la même architecture (propagation $\tilde{\boldsymbol{D}}^{-1/2}\tilde{\boldsymbol{A}}\tilde{\boldsymbol{D}}^{-1/2}$, ReLU, dropout 0,3, Adam, $L_2 = 5 \cdot 10^{-4}$) car PyTorch Geometric n'était pas disponible dans l'environnement de génération. Des écarts mineurs sont possibles avec le Code 2.3

(initialisation, graine). Avec environ 30 UE de test par scénario, la variance est élevée : d'où la moyenne sur 10 scénarios.

2.4 Analyse critique et interprétation

2.4.1 Capacité de classification

L'analyse doit porter sur la matrice de confusion (Figure 2.3) et les métriques par classe. Les erreurs les plus probables concernent la classe intermédiaire ("moyenne"), située entre deux seuils : une petite erreur d'estimation du SINR suffit à changer de classe. La classe "mauvaise" est généralement la plus rare et la plus importante pour l'opérateur ; on privilégiera donc son rappel, éventuellement avec une pondération des classes dans la loss.

2.4.2 Résultats obtenus

Sur le scénario de référence (graine 42), le GCN atteint 70 % d'accuracy test (rappel de 83 % sur la classe "mauvaise", 40 % sur "moyenne", 88 % sur "bonne"), contre 76,7 % pour le MLP et 70 % pour la règle à seuils. Sur 10 scénarios (Table 2.2), **le GCN ne surpasse pas les références** dans notre configuration ; la règle à seuils et le MLP sont meilleurs en moyenne, avec des écarts-types élevés qui interdisent toute conclusion tranchée. Ce résultat est honnête et instructif : la puissance reçue du gNB servant explique déjà l'essentiel du SINR, et l'information de voisinage n'apporte qu'un second ordre. Les courbes (Figure 2.2) montrent que le GCN est encore en cours de convergence à 200 époques (loss encore décroissante, pas d'écart marqué entre entraînement et validation) : il s'agit plutôt de **sous-apprentissage** que de surapprentissage. Pistes : plus d'époques, taux d'apprentissage plus élevé, plus d'UE, normalisation des degrés. La suppression des arêtes gNB–gNB dégrade légèrement le GCN (65,0 → 62,0 %), ce qui va dans le sens attendu mais reste dans la marge d'incertitude.

2.4.3 Surapprentissage

Le jeu d'entraînement est petit (environ 140 UE) et le modèle possède quelques milliers de paramètres. Un écart croissant entre l'accuracy d'entraînement et de validation, ou une remontée de la loss de validation, indique un surapprentissage. Les remèdes mis en œuvre ou possibles : dropout, régularisation L_2 (`weight_decay`), sélection du meilleur état sur la validation, arrêt précoce, augmentation du nombre de scénarios générés.

2.4.4 Impact de la topologie

Deux expériences d'ablation permettent d'isoler l'apport de la structure :

1. **GCN vs MLP** : mêmes caractéristiques, même profondeur ; seule l'agrégation de voisinage change. Un gain du GCN indique que la charge des cellules voisines et la densité d'UE portent une information utile sur l'interférence.
2. **Perturbation de la topologie** : supprimer les arêtes gNB–gNB, ou les remplacer par des arêtes aléatoires, et mesurer la dégradation.

Un gain n'est pas garanti : si le bruit d'ombrage domine, ou si l'interférence est faible (réseau peu chargé), le MLP peut égaler le GCN. Cette conclusion nuancée serait elle-même un résultat valide.

2.4.5 Limites

— Modèle radio simplifié et données synthétiques ; la validité externe est limitée (une extension vers ns-3/5G-LENA est prévue en perspective) ;

— Une bonne accuracy de classification n'implique pas nécessairement une amélioration du débit ou de la latence ;

— Le SINR de l'UE dépend des charges, elles-mêmes fonction de l'association : la cible est donc partiellement circulaire, ce qui est un défaut assumé du modèle pédagogique ;

— La comparaison "meilleur SINR" du guide est réalisée ici par la règle à seuils sur P_r ; pour l'extension *prédiction du meilleur gNB*, la règle classique consistera à choisir $\arg\max_j \mathrm{SINR}_{ij}$ et sera comparée à une prédiction d'arête (*link prediction*) par GNN.

Conclusion du chapitre

Nous avons construit un pipeline reproductible, sans fuite de données, allant du scénario radio au GCN évalué et comparé à deux références. Le chapitre suivant replace ce résultat dans une comparaison plus générale entre CNN et GNN.

Chapitre 3

Comparatif : CNN vs GNN

3.1 Tableau comparatif

Table 3.1 – Comparaison structurée entre réseaux convolutifs (CNN) et réseaux sur graphes (GNN).

Critère	CNN	GNN
Type de données d'entrée	Grilles régulières (images, vidéos, signaux 1D) de taille fixe ou redimensionnable	Graphes irréguliers : nombre de nœuds et d'arêtes variable, attributs de nœuds et d'arêtes
Type de voisinage	Fixe et euclidien : noyau $k \times k$ ordonné, même taille partout	Variable et non euclidien : degré quelconque, voisins sans ordre, défini par E
Invariance aux permutations	Non invariant : permuter les pixels change la sortie	Invariant (équivariant par nœud, invariant après *readout*) grâce à l'agrégation symétrique
Partage de paramètres	Par translation (noyau glissant)	Entre tous les nœuds (même $\boldsymbol{W}$)
Opérateur de base	Convolution discrète	Message passing / convolution spectrale ou spatiale sur graphe
Domaines typiques	Vision par ordinateur, imagerie médicale, reconnaissance de formes	Réseaux sociaux, molécules et protéines, graphes de connaissances, systèmes de recommandation, réseaux de télécommunications

3.2 Analyse détaillée

3.2.1 Le CNN comme cas particulier de GNN

Une image peut être vue comme un graphe dont les nœuds sont les pixels, reliés à leurs 8 voisins. La convolution 3×3 est alors un message passing à poids *positionnels* (un poids par position relative). Le GNN renonce à cet ordre pour obtenir l'invariance, au prix d'une perte de l'information de position relative, sauf si elle est ajoutée comme attribut d'arête.

3.2.2 Forces et limites du CNN

Forces : biais inductifs très adaptés aux images (localité, invariance à la translation), opérations denses très optimisées sur GPU, écosystème mature (architectures, modèles pré-entraînés). **Limites :** inadapté aux structures irrégulières ; champ récepteur limité en profondeur ; non invariant aux rotations sans augmentation de données ; pas de modélisation explicite de relations arbitraires entre éléments.

3.2.3 Forces et limites du GNN

Forces : manipulation naturelle de données relationnelles, généralisation à des graphes de tailles différentes, prise en compte explicite de la topologie, applicabilité inductive (GraphSAGE). **Limites :** sur-lissage et goulot d'étranglement [11, 12] ; expressivité bornée par le test de Weisfeiler–Leman [8] ; calcul irrégulier (accès mémoire non contigus) et passage à l'échelle difficile sur les graphes massifs ; forte dépendance à la qualité du graphe défini (choix des arêtes) ; pas de standard d'architecture aussi établi que pour les CNN.

Quel paradigme choisir ?

Le choix dépend de la nature des données : si une grille régulière existe, le CNN est le choix par défaut. Si la relation entre entités est le cœur du problème, le GNN est justifié. Le projet 5G illustre ce point : positions et mesures d'UE sont irrégulières, et le voisinage radio n'a pas de structure de grille.

Chapitre 4

BONUS : Synergie entre LLM et GNN

4.1 Pourquoi combiner LLM et GNN ?

Les grands modèles de langage (LLM), fondés sur l'architecture Transformer [17], possèdent de fortes capacités de compréhension, de raisonnement textuel et de génération, mais :

— ils gèrent mal la **structure relationnelle complexe** (multi-sauts, grande cardinalité), car un graphe sérialisé en texte est long et son ordre est arbitraire ;

— ils peuvent **halluciner** des faits non ancrés dans une base de connaissances ;

— leur fenêtre de contexte est limitée.

Inversement, les GNN encodent efficacement la topologie, mais :

— leurs entrées sont des vecteurs numériques, avec une compréhension limitée du **texte** associé aux nœuds (titres, descriptions, documents) ;

— ils manquent de capacité de **raisonnement global** et de généralisation en langage naturel ;

— ils dépendent d'étiquettes et ne disposent pas de connaissances générales pré-entraînées.

Leur complémentarité est analysée dans [20] et dans les revues récentes sur LLM et graphes [21].

4.2 Cas d'usage majeurs

4.2.1 GraphRAG

La génération augmentée par récupération (RAG) [18] ancre les réponses d'un LLM dans des documents externes. Le **GraphRAG** remplace ou complète l'index vectoriel par un **graphe de connaissances** : entités et relations extraites (souvent par LLM), découpage en communautés, résumés hiérarchiques [19]. Le récupérateur peut être un GNN qui score les nœuds et sous-graphes pertinents, puis fournit au LLM un sous-graphe *textualisé* ou encodé en jetons souples, comme dans G-Retriever [23]. Avantages : réponses aux questions multi-sauts et globales, traçabilité des sources, réduction des hallucinations.

4.2.2 Alignement texte–graphe

Un encodeur de graphe (GNN) et un encodeur de texte (LLM ou petit modèle de langage) sont alignés dans un espace latent commun, par apprentissage contrastif ou par un projecteur appris vers l'espace d'*embeddings* du LLM. Applications : récupération de nœuds par requête textuelle, classification zéro-coup de nœuds, question-réponse sur graphes, et annotation de molécules par du texte.

4.2.3 Amélioration des embeddings de nœuds par descriptions textuelles

Pour les graphes à attributs textuels (articles cités, produits, équipements réseau), un LLM génère des explications, résumés ou pseudo-étiquettes enrichissant chaque nœud ; un modèle de langage plus petit les convertit en caractéristiques denses qui alimentent ensuite un GNN [22]. Dans notre contexte 5G, on pourrait convertir des journaux d'incidents ou des descriptions de cellules ("cellule rurale, forte pente, incidents de congestion répétés") en vecteurs, ajoutés aux caractéristiques radio de la Table 2.1.

4.3 Architecture type de la synergie

Description du Schéma 3 – Architecture LLM + GNN

Placement : Figure 4.1. **Flux de gauche à droite.** (1) Source de données : documents textuels et graphe (nœuds, arêtes). (2) Deux voies parallèles : *voie texte* (encodeur LLM → *embeddings* textuels de nœuds) et *voie graphe* (GNN → *embeddings* structurels). (3) Un module de *fusion/alignement* (concaténation, attention croisée ou perte contrastive). (4) Un récupérateur de sous-graphe pertinent pour la requête. (5) Un LLM générateur qui reçoit la requête et le sous-graphe (textualisé ou en jetons) et produit la réponse. Une flèche de retour indique le fine-tuning conjoint (par exemple adaptateurs LoRA).

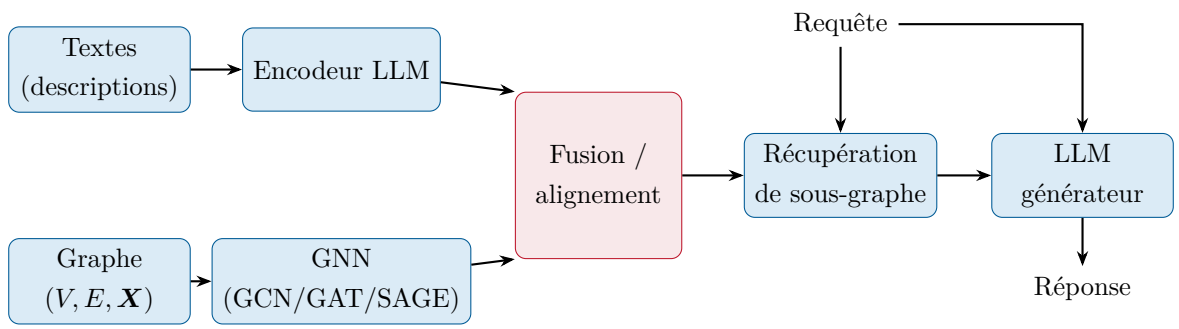


Figure 4.1 – Architecture type d'une synergie LLM + GNN pour le GraphRAG et l'alignement texte–graphe (la figure peut être réduite avec `\resizebox`).

4.4 Exemple de code : fusion d'embeddings textuels et structurels

```python
import torch
from torch_geometric.nn import GCNConv

class TextAugmentedGCN(torch.nn.Module):
    """text_emb : tenseur [N, d_txt] precalcule (ex. sentence-transformer / LLM)
       x        : caracteristiques numeriques [N, d_num]"""
    def __init__(self, d_num, d_txt, hidden=32, out_ch=3):
        super().__init__()
        self.txt_proj = torch.nn.Linear(d_txt, 16)       # reduction de dimension du texte
        self.conv1 = GCNConv(d_num + 16, hidden)
        self.conv2 = GCNConv(hidden, out_ch)

    def forward(self, x, text_emb, edge_index):
        z = torch.cat([x, torch.relu(self.txt_proj(text_emb))], dim=1)
        z = torch.relu(self.conv1(z, edge_index))
        return self.conv2(z, edge_index)
```

Code 4.1 – Concaténation d'embeddings textuels (précalculés par un LLM) aux caractéristiques de nœuds avant un GCN.

4.5 Limites et défis

Les principaux défis sont le coût d'inférence des LLM, la qualité de l'extraction automatique du graphe (erreurs d'entités et de relations), la perte d'information lors de la textualisation, l'alignement des espaces de représentation et l'évaluation (peu de bancs d'essai standardisés). Pour la 5G, la confidentialité des données d'exploitation impose d'étudier des déploiements locaux.

Conclusion et Perspectives

Bilan

Ce travail a présenté les fondements et le fonctionnement des GNN (message passing, fonctions d'agrégation, GCN, GAT, GraphSAGE), puis les a appliqués à un cas d'étude de connectivité 5G : modélisation d'un réseau UE–gNB en graphe, génération de données radio cohérentes, implémentation d'un GCN à deux couches avec PyTorch Geometric, et évaluation sans fuite de données face à deux références ne reposant pas sur la topologie. Le comparatif CNN vs GNN a clarifié le domaine de pertinence de chaque paradigme, et le chapitre bonus a montré comment LLM et GNN se complètent.

Perspectives

— **Prédiction du meilleur gNB** par prédiction d'arêtes (*link prediction*), comparée à la règle "meilleur SINR" ;

— **Mobilité des UE** et graphes dynamiques (GNN temporels), événements de congestion ;

— **Comparaison GCN / GraphSAGE / GAT**, avec attention interprétable pour identifier les cellules interférentes ;

— **Régression** du SINR ou du débit plutôt que classification, avec mesure d'impact sur le débit et la latence ;

— **Données plus réalistes** avec ns-3/5G-LENA ou les modèles 3GPP [16], et jeux de données opérateur ;

— **Synergie LLM + GNN** : GraphRAG sur les documents de réglementation et de planification radio, assistance à l'exploitation réseau.

Bibliographie

[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 1024–1034.

[4] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1263–1272.

[5] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.

[6] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning : Going beyond Euclidean data," *IEEE Signal Process. Mag.*, vol. 34, no. 4, pp. 18–42, Jul. 2017.

[7] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.

[8] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks ?" in *Proc. ICLR*, 2019.

[9] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 29, 2016.

[10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[11] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018.

[12] U. Alon and E. Yahav, "On the bottleneck of graph neural networks and its practical implications," in *Proc. ICLR*, 2021.

[13] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks ?" in *Proc. ICLR*, 2022.

[14] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[15] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, "Unveiling the potential of graph neural networks for network modeling and optimization in SDN," in *Proc. ACM Symp. SDN Research (SOSR)*, 2019, pp. 140–151.

[16] 3GPP, "Study on channel model for frequencies from 0.5 to 100 GHz," 3rd Generation Partnership Project, Tech. Rep. TR 38.901.

[17] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[18] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[19] D. Edge *et al.*, "From local to global : A graph RAG approach to query-focused summarization," arXiv :2404.16130, 2024.

[20] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs : A roadmap," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3580–3599, Jul. 2024.

[21] Z. Chen *et al.*, "Exploring the potential of large language models (LLMs) in learning on graphs," *ACM SIGKDD Explor. Newsl.*, vol. 25, no. 2, pp. 42–61, 2024.

[22] X. He *et al.*, "Harnessing explanations : LLM-to-LM interpreter for enhanced text-attributed graph representation learning," in *Proc. ICLR*, 2024.

[23] X. He *et al.*, "G-Retriever : Retrieval-augmented generation for textual graph understanding and question answering," arXiv :2402.07630, 2024.